fn sample_poisson_0_1

OpenDP

March 16, 2026

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `fn sample_poisson_0_1` in `mod.rs`. This proof uses a geometric proposal distribution on $\mathbb{N}_0$ and accepts with probability $1/k!$ to realize the exact Poisson law on the interval $[0, 1)$.

1 Hoare Triple

Precondition

Compiler-verified

- Argument `lambda` is of type `RBig`, a bignum rational

User-verified

$0 \leq$ `lambda` < 1

Implementation

```
def sample_poisson_0_1(lmbda):
    if lmbda == 0:
        return 0

    while True:
        k = 0
        while sample_bernoulli_rational(lmbda):  #
            k += 1

        i = 2
        accepted = True
        while i <= k:
            if not sample_bernoulli_rational(1 / i):  #
                accepted = False
                break
            i += 1

        if accepted:  #
            return k
```

Postcondition

Theorem 1.1. For any setting of the input parameter `lambda` such that the given preconditions hold, `sample_poisson_0_1` either returns `Err(e)` due to a lack of system entropy, or `Ok(out)`, where `out` is distributed as $Poisson($`lambda`$)$.

2 Proof

Lemma 2.1. `sample_poisson_0_1` only returns `Err(e)` when there is a lack of system entropy.

Proof. If `lambda` $= 0$, the function returns `Ok(0)` immediately.

Otherwise, the function only invokes `sample_bernoulli_rational` on lines 7 and 13. Since the precondition gives $0 <$ `lambda` < 1, the first call is valid. On line 13, the argument is $1/i$ for an integer $i \geq 2$, so this probability also lies in $(0, 1)$. By the definition of `sample_bernoulli_rational`, an error can only arise from a lack of system entropy. Therefore `sample_poisson_0_1` only returns `Err(e)` when there is a lack of system entropy. □

We now condition on not returning an error and fix any iteration of the outer loop.

Lemma 2.2. If `k` is the value after line 7, then for every $k \in \mathbb{N}_0$,

$$\Pr[K = k] = (1 - \lambda)\lambda^k.$$

Proof. The loop on line 7 increments `k` once for each consecutive success of a Bernoulli random variable with success probability λ, and stops at the first failure. Therefore `k` is geometric on $\mathbb{N}_0$ with

$$\Pr[K = k] = (1 - \lambda)\lambda^k.$$

□

Lemma 2.3. Conditional on $K = k$, the probability that the algorithm returns k in a fixed outer-loop iteration is $1/k!$.

Proof. If $k = 0$ or $k = 1$, the acceptance loop does not reject and the return probability is $1 = 1/k!$.

If $k \geq 2$, line 13 performs independent Bernoulli draws with probabilities $1/2, 1/3, \ldots, 1/k$, and returns k only if all of them succeed. Thus

$$\Pr[\text{return } k \mid K = k] = \prod_{i=2}^{k} \frac{1}{i} = \frac{1}{k!}.$$

□

Lemma 2.4. For every $k \in \mathbb{N}_0$, the probability that a fixed outer-loop iteration returns k is

$$(1 - \lambda)\frac{\lambda^k}{k!}.$$

Proof. By Lemma 2.2 and Lemma 2.3,

$$\Pr[\text{one iteration returns } k] = \Pr[K = k]\Pr[\text{return } k \mid K = k] = (1 - \lambda)\lambda^k \cdot \frac{1}{k!}.$$

□

Lemma 2.5. The probability that a fixed outer-loop iteration returns some value is

$$(1 - \lambda)e^{\lambda}.$$

Proof. By Lemma 2.4,

$$\Pr[\text{one iteration returns a value}] = \sum_{k=0}^{\infty}(1 - \lambda)\frac{\lambda^k}{k!} = (1 - \lambda)\sum_{k=0}^{\infty}\frac{\lambda^k}{k!} = (1 - \lambda)e^{\lambda}.$$

□

Lemma 2.6. If the outcome of `sample_poisson_0_1` is `Ok(out)`, then `out` is distributed as $Poisson(\texttt{lambda})$.

Proof. Let

$$r_k := (1-\lambda)\frac{\lambda^k}{k!} \quad \text{and} \quad a := (1-\lambda)e^{\lambda}.$$

By Lemma 2.4, a fixed outer-loop iteration returns k with probability r_k. By Lemma 2.5, a fixed outer-loop iteration returns some value with probability a.

Since outer-loop iterations are independent, the probability that the algorithm returns k on iteration t is $(1-a)^{t-1}r_k$. Therefore

$$\begin{aligned}
\Pr[\texttt{out} = k] &= \sum_{t=1}^{\infty}(1-a)^{t-1}r_k \\
&= \frac{r_k}{a} \\
&= \frac{(1-\lambda)\lambda^k/k!}{(1-\lambda)e^{\lambda}} \\
&= e^{-\lambda}\frac{\lambda^k}{k!}.
\end{aligned}$$

Thus `out` is distributed as $Poisson(\texttt{lambda})$. □

Proof of Theorem 1.1. Holds by Lemma 2.1 and Lemma 2.6. □